

《微机原理实验》实验报告

实验名称: 数模转换实验 指导教师: 肖山林

姓名: 徐睿琳 学号: 23342107 专业/班级: 微电子/三班 分组序号: A412-A05

实验日期: 2025.12.15 实验地点: 教学楼 A412 是否调课/补课: 否 成绩: _____

目录

1 实验内容与设计	1
1.1 D/A 数模转换实验	1
1.1.1 三角波生成实验	1
1.1.2 锯齿波生成实验	2
1.1.3 正弦波生成实验	3
1.2 A/D 模数转换实验	3
2 实验结果	4
2.1 D/A 数模转换实验	4
2.2 A/D 模数转换实验	5
3 思考与总结	6
3.1 关于 D/A 转换与波形拟合精度的思考	6
3.2 关于 A/D 转换控制方式的改进探讨	7
3.3 微机接口实验的综合体会	7
附录 A 脚本代码	8

1 实验内容与设计

1.1 D/A 数模转换实验

1.1.1 三角波生成实验

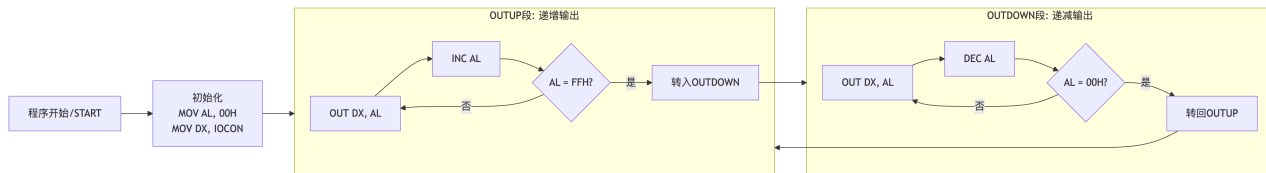


Figure 1: 三角波生成逻辑流程图

```
1  CODE SEGMENT
2  ASSUME CS:CODE
3  IOCON EQU 0B000H
4
5  START:
6      MOV AL, 00H
7      MOV DX, IOCON
8
9  OUTUP:
```

```

10     OUT DX, AL
11     INC AL
12     CMP AL, 0FFH      ; 判断是否达到最大值
13     JE OUTDOWN        ; 达到则跳转递减段
14     JMP OUTUP
15
16 OUTDOWN:
17     DEC AL
18     OUT DX, AL
19     CMP AL, 00H      ; 判断是否减到0
20     JE OUTUP         ; 为0则跳回递增段
21     JMP OUTDOWN
22
23 CODE ENDS
24 END START

```

1.1.2 锯齿波生成实验

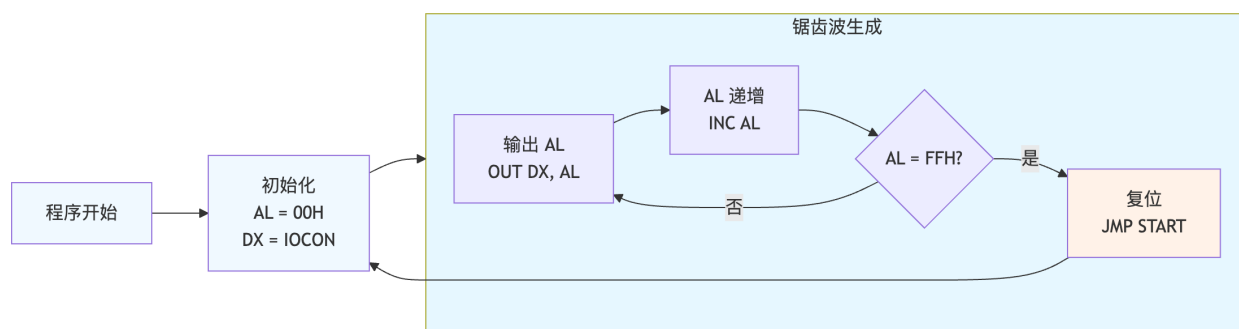


Figure 2: 锯齿波生成逻辑流程图

```

1     CODE SEGMENT
2     ASSUME CS:CODE
3     IOCON EQU 0B000H
4
5     START:
6         MOV AL, 00H
7         MOV DX, IOCON
8
9     OUTUP:
10        OUT DX, AL
11        INC AL
12        CMP AL, 0FFH
13        JE RESET
14        JMP OUTUP
15
16    RESET:
17        JMP START
18
19    CODE ENDS
20    END START

```

1.1.3 正弦波生成实验

第一步：生成正弦波数据表

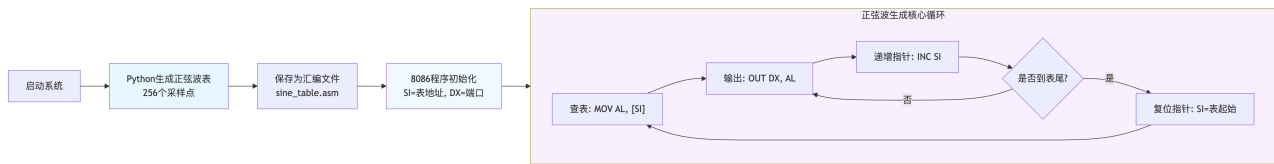


Figure 3: 正弦波生成逻辑流程图

```
python sin_wave_generator.py
```

脚本代码见附录 A。

第二步：根据正弦数据表生成正弦波

```

1  CODE SEGMENT
2      ASSUME CS:CODE, DS:DATA
3
4  START:
5      MOV AX, DATA
6      MOV DS, AX
7      MOV SI, OFFSET SINE_TABLE
8      MOV DX, IOCON
9
10 SINE_LOOP:
11     MOV AL, [SI]           ; 查表获取正弦值
12     OUT DX, AL             ; 输出到端口
13
14     INC SI                 ; 指针递增
15     CMP SI, OFFSET SINE_TABLE_END
16     JB CONTINUE           ; 未到表尾则继续
17     MOV SI, OFFSET SINE_TABLE ; 重置指针
18
19 CONTINUE:
20     JMP SINE_LOOP
21
22 CODE ENDS
23
24 ; 正弦波表（替换成Python脚本生成的版本）
25 DATA SEGMENT
26 SINE_TABLE DB 128, 131, 134, 137, 140, 143, 146, 149
27             DB 152, 155, 158, 161, 164, 167, 170, 173
28             ; ... 省略中间240个数据
29             DB 125, 122, 119, 116, 113, 110, 107, 104
30             DB 101, 98, 95, 92, 89, 86, 83, 80
31 SINE_TABLE_END LABEL BYTE
32 DATA ENDS
33 END START
  
```

1.2 A/D 模数转换实验

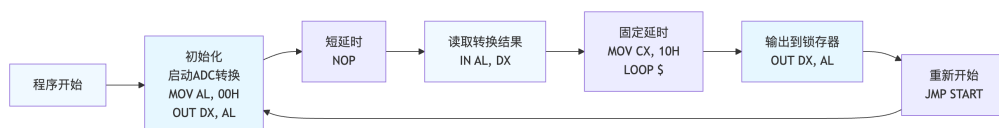


Figure 4: A/D 模数转换实验逻辑流程图

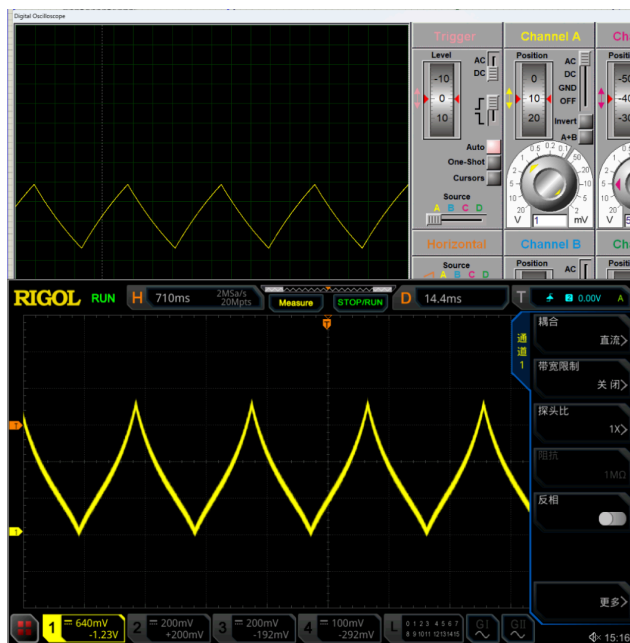
```

1  CODE SEGMENT
2      ASSUME CS:CODE
3
4      AD0809 EQU 0E002H
5      OUT373 EQU 8000H
6
7  START:
8      MOV AL, 00H
9      MOV DX, AD0809
10     OUT DX, AL      ; 启动ADC0809转换
11
12     NOP
13
14     IN AL, DX        ; 读取ADC0809转换结果
15     MOV CX, 10H
16     LOOP $           ; 延时等待
17     MOV DX, OUT373
18     OUT DX, AL       ; 结果输出到74LS373锁存器
19
20     JMP START        ; 重新启动下一次转换
21
22 CODE ENDS
23 END START

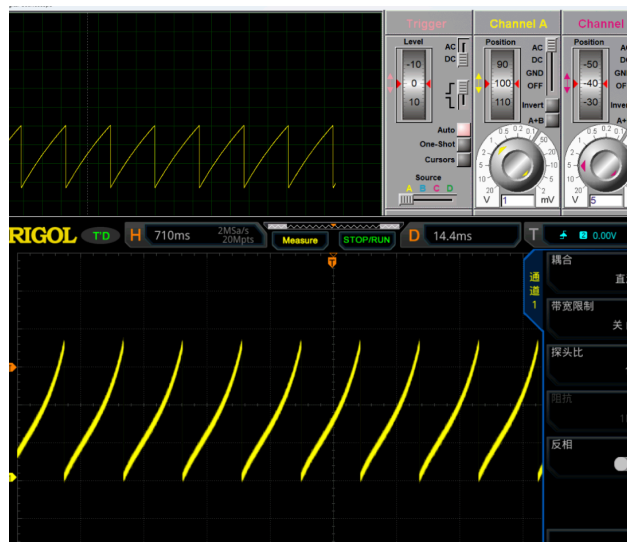
```

2 实验结果

2.1 D/A 数模转换实验

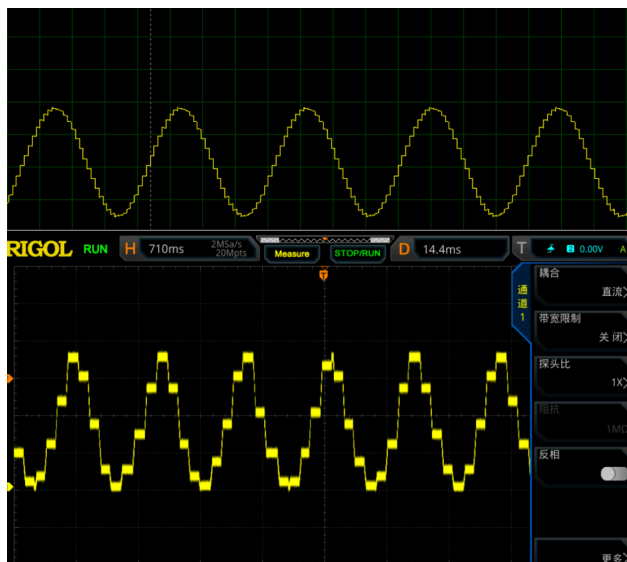


(a) 三角波仿真软件-示波器截图

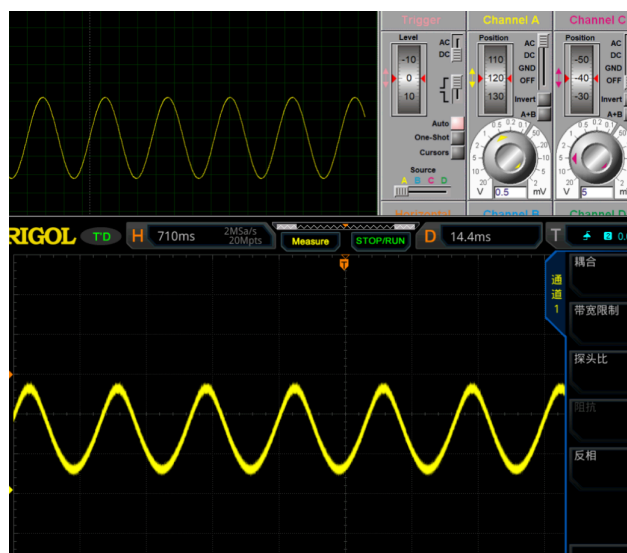


(b) 锯齿波仿真软件-示波器截图

根据查表法与自动脚本配合，理论上可以生成任意精度的正弦波形状及其他函数波形，实际效果与数据点数量成正比。实验中发现，数据点数量越多，生成的正弦波形状越接近理想曲线，失真越小。

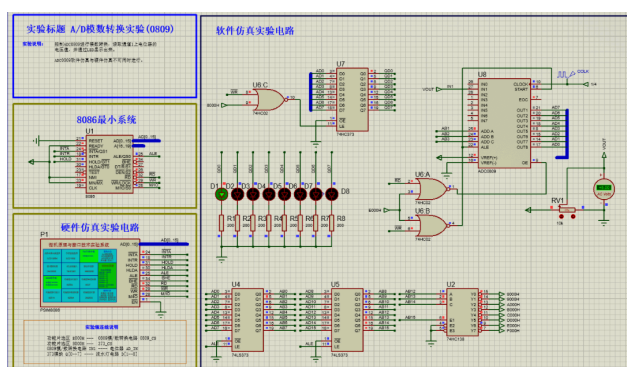


(a) 正弦波 (60 点拟合) 仿真软件-示波器截图

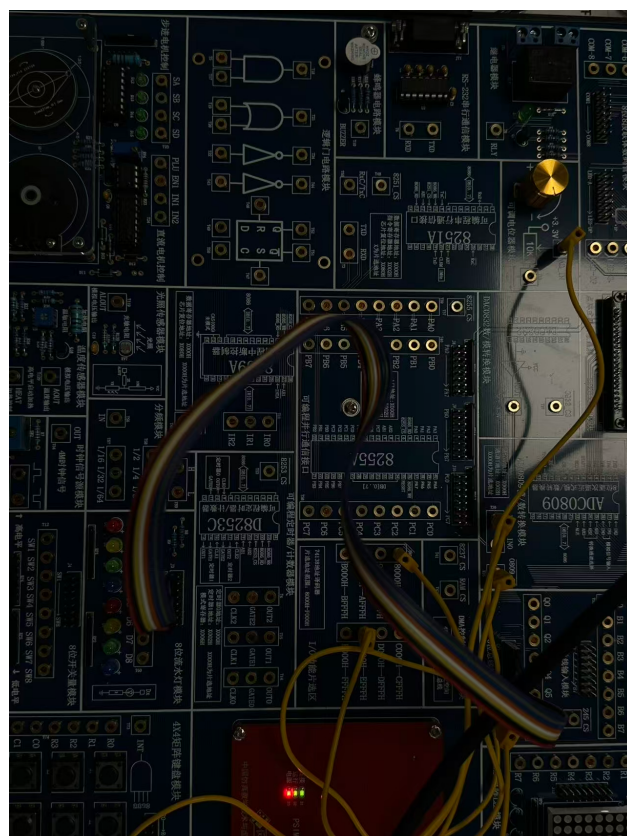


(b) 正弦波 (256 点拟合) 仿真软件-示波器截图

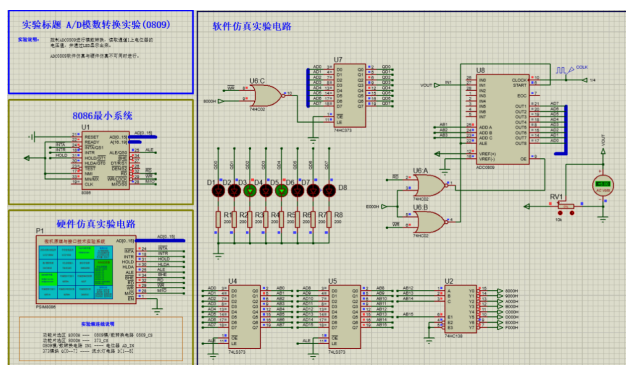
2.2 A/D 模数转换实验



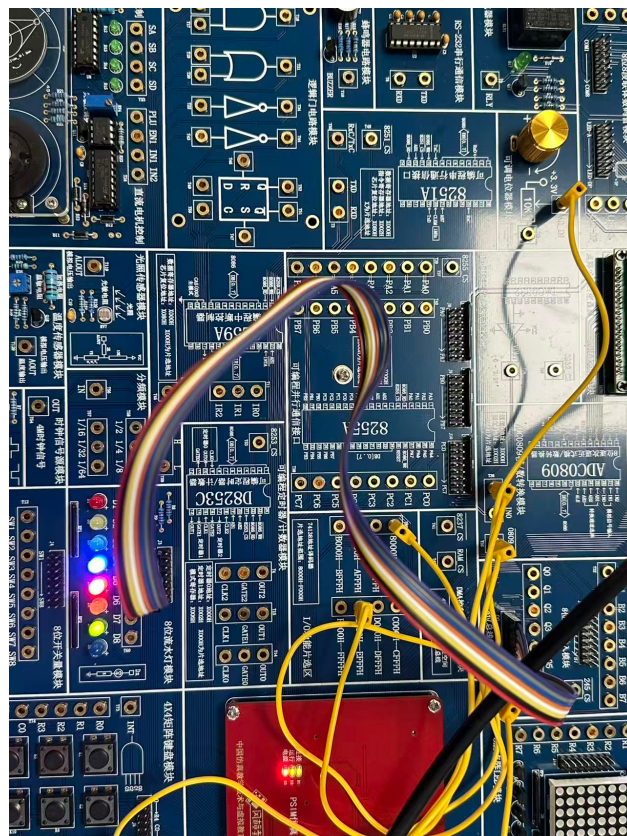
(a) 电位器最小阻值仿真软件显示



(b) 电位器最小阻值 LED 灯珠显示



(a) 电位器中间阻值时仿真软件显示



(b) 电位器中间阻值时 LED 灯珠显示

3 思考与总结

3.1 关于 D/A 转换与波形拟合精度的思考

本次实验通过 D/A 转换器生成了三角波、锯齿波和正弦波。对于三角波和锯齿波，其生成逻辑是线性的，通过简单的寄存器加减指令即可实现。而对于正弦波，实验采用了查表法 (Look-Up Table, LUT)，这是一种典型的“以空间换时间”的算法策略。

这两种策略在汇编层面的对比如下：

```

1 ; 1. 线性生成 (锯齿波/三角波)
2 INC AL ; 依赖 CPU 运算指令直接生成数据
3 OUT DX, AL ; 实时性强，但波形单一
4
5 ; 2. 查表法 (正弦波)
6 MOV AL, [SI] ; 依赖预先计算好的内存数据
7 OUT DX, AL ; 灵活性高，可生成任意波形
8 INC SI
  
```

在实验中对比了 60 点拟合与 256 点拟合的效果，可以明显观察到点数越多，波形越平滑。这引出了 D/A 转换中的两个关键指标：**分辨率**和**采样率**。

- **幅度量化误差**：受限 DAC0832 的 8 位精度，输出电压只能在 $0 \sim V_{ref}$ 之间分为 256 个离散电平，这导致波形在纵轴上必然存在锯齿状的量化噪声。
- **时间离散化**：CPU 输出数据的速度决定了横轴的时间分辨率。

拓展思考：在实际工程应用中，为了获得更纯净的模拟信号，通常会在 DAC 输出端级联一个**低通滤波器 (LPF)**，用于滤除由阶梯波产生的高频谐波分量。

3.2 关于 A/D 转换控制方式的改进探讨

在 A/D 模数转换实验中，代码使用了软件延时的方式来等待 ADC0809 完成转换。这种方式被称为**盲延时**，其缺点是 CPU 效率低下且依赖经验值。

改进方案：利用 ADC0809 的 **EOC (End of Conversion)** 信号进行查询控制。相比于当前的延时代码，查询方式能自适应硬件速度：

```
1 ; 当前方式：盲延时（效率低）
2 MOV CX, 10H
3 LOOP $ ; CPU 空转等待
4
5 ; 改进方式：查询 EOC 状态（假设状态口为 STATUS_PORT）
6 WAIT_EOC:
7     IN AL, STATUS_PORT
8     TEST AL, 80H ; 检测 EOC 标志位（假设在最高位）
9     JZ WAIT_EOC ; 若未完成则继续等待
10    IN AL, DATA_PORT ; 完成后立即读取
```

此外，更高级的控制方式是**中断方式**：将 EOC 信号连接至 CPU 的中断请求引脚（如 8259A 的 IRQ），转换结束后触发中断服务程序读取数据。这种方式下 CPU 利用率最高，适合多任务实时系统。

3.3 微机接口实验的综合体会

通过本次实验，深刻理解了“端口”在微机系统中的核心地位。‘OUT’和‘IN’指令不仅仅是数据的传输，往往还伴随着硬件控制信号的触发。

例如，向 ADC0809 的通道地址写入数据这一操作：

```
1 OUT DX, AL ; 既传输了通道号(AL)，又产生了 WR 写信号启动了转换
```

这说明在接口技术中，软件指令与硬件时序是紧密耦合的。这种从“数字代码”到“模拟物理量”再回到“数字显示”的闭环过程，是微机原理与接口技术的精髓所在。

附录 A 脚本代码

```
1  import math
2
3  def generate_sine_table(num_points=256, amplitude=127.5, offset=127.5):
4      sine_table = []
5      for i in range(num_points):
6          sine_value = math.sin(2 * math.pi * i / num_points)
7          dac_value = int(amplitude * sine_value + offset)
8          dac_value = max(0, min(255, dac_value))
9          sine_table.append(dac_value)
10     return sine_table
11
12 def save_as_assembly_table(sine_table, filename="sine_table.asm"):
13     with open(filename, 'w') as f:
14         f.write("SINE_TABLE LABEL BYTE\n")
15         for i, value in enumerate(sine_table):
16             if i % 16 == 0:
17                 f.write(f"\tDB {value}")
18             else:
19                 f.write(f", {value}")
20             if i % 16 == 15 or i == len(sine_table) - 1:
21                 f.write("\n")
22
23 def main():
24     sine_table = generate_sine_table()
25     save_as_assembly_table(sine_table)
26     print(f"表大小: {len(sine_table)} 字节")
27     print("前10个正弦波值:")
28     for i in range(10):
29         print(f"位置 {i}: {sine_table[i]} (0x{sine_table[i]:02X})")
30     print(f"最小值: {min(sine_table)}")
31     print(f"最大值: {max(sine_table)}")
32     print(f"平均值: {sum(sine_table)/len(sine_table):.1f}")
33
34 if __name__ == "__main__":
35     main()
```